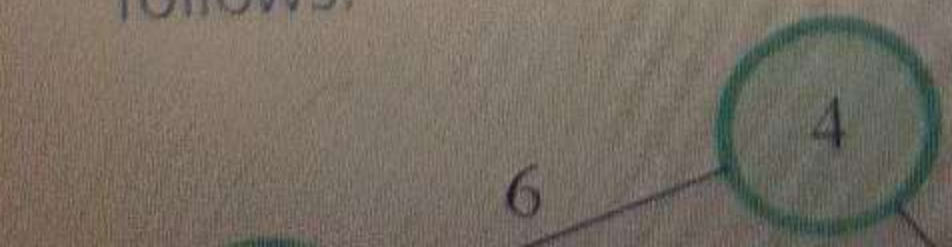
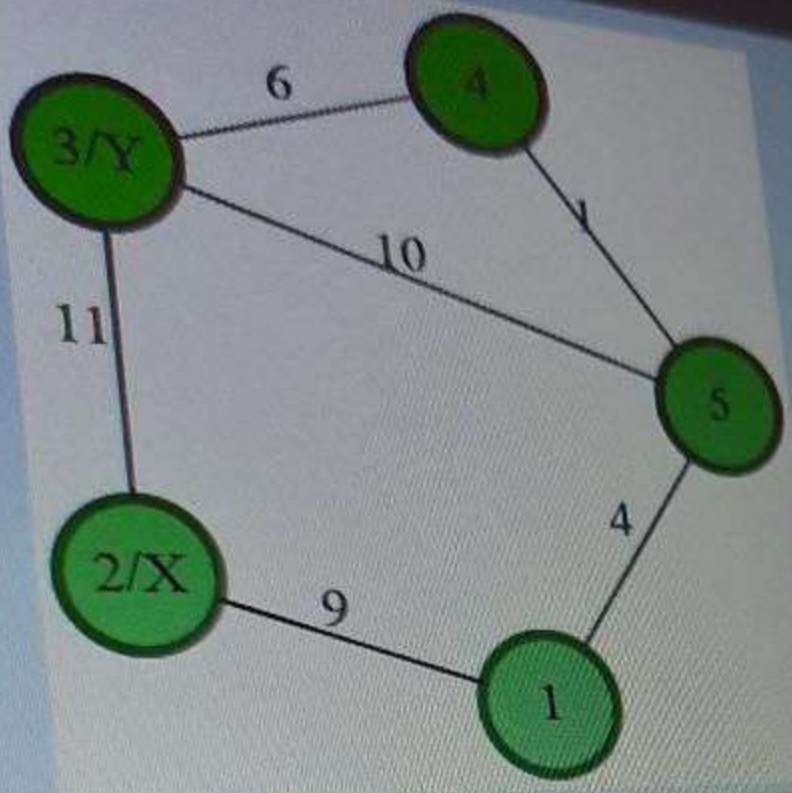
1. Two Junctions

Jack has two errands he needs to run before he can go to school, and they need to be done in order. Time is getting tight, so he needs to take the quickest route. Determine the fastest route from where he starts, through the two errand locations, and on to his school. The city will be represented as an undirected graph of nodes labeled incrementally from *1*.

For example, there are *g_nodes = 5*, labeled *1* to *g_nodes*, inclusive. Nodes represent junctions or milestones along a route. Jack's route always starts at node *1* and the school is always at node *g_nodes*, in this case, node *5*. The starting nodes for roads, *g_from = [1,2,3,4,5,3]*, ending nodes *g_to = [2, 3, 4, 5, 1, 5]* and times to travel, given as *g_weight = [9, 11, 6, 1, 4, 10]*, are aligned by index . In this example, he starts at node *1*, has to visit *x = node 2* then *y = node 3*, in that order, before going to the school at node *5*. A graphical representation follows.





There are two paths he can take to get to school: $1 \rightarrow 2 \rightarrow 3 \rightarrow 5$ and $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$. The first route takes $9 + 11 + 10 = 30$ travel time while the second route only takes $9 + 11 + 6 + 1 = 27$. The function should return 27 in this case.

Note: Jack can visit any node multiple times.

Function Description

Complete the function *minCostPath* in the editor below. The function must return the time it takes to travel the shortest path from node *1* through *x* and then *y*, arriving finally at node *g_nodes*.

minCostPath has the following parameter(s):

g_from[g_from[0],...g_from[g_edges-1]]: an array of integers that represent edge start nodes

g_to[g_to[0],...g_to[g_edges-1]]: an array of integers that represent edge end nodes

g_weight[g_weight[0],...g_weight[g_edges-1]]: an

Language: C++14

```
/*
 * Complete the 'minCostPath' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts following parameters:
 *  1. WEIGHTED_INTEGER_GRAPH g
 *  2. INTEGER x
 *  3. INTEGER y
 */

/*
 * For the weighted graph, <name>:
 *
 * 1. The number of nodes is <name>_nodes.
 * 2. The number of edges is <name>_edges.
 * 3. An edge exists between <name>_from[i] and
 *
 */

int minCostPath(int g_nodes, int
{

}

int main()
```

Test Results Custom Input

function minCostPath in the editor below. The function must return the time it takes to travel the shortest path from node 1 through x and then y, arriving finally at node g_nodes.

minCostPath has the following parameter(s):

g_from[g_from[0],...g_from[g_edges-1]]: an array of integers that represent edge start nodes

g_to[g_to[0],...g_to[g_edges-1]]: an array of integers that represent edge end nodes

g_weight[g_weight[0],...g_weight[g_edges-1]]: an array of integers that represent time to travel an edge, its weight

x: an integer, the first node that must be on the path

y: an integer, the second node that must be on the path

Constraints

- $4 \le g\_nodes \le 10^5$
- $4 \le g\_edges \le min(10^5, (g\_nodes \times (g\_nodes - 1)) / 2)$
- $1 \le g\_weight[i] \le 10^3$
- $2 \le x \le g\_nodes-1$
- $2 \le y \le g\_nodes-1$

▶ Input Format For Custom Testing

▼ Sample Case 0

Language: C++14

Test Results

Search

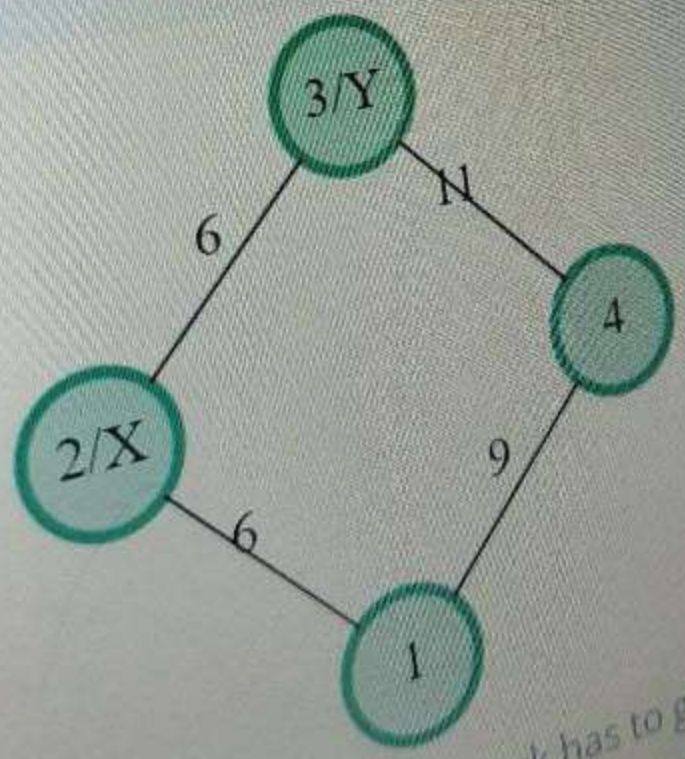
Sample Input 0

```
4 5
1 2 6
1 4 9
2 4 10
2 3 6
3 4 11
2
3
```

Sample Output 0

```
23
```

Explanation 0



For the given graph Jack has to go from junction *1* to junction *g_nodes* and his path should include junctions *2* and *3* in that order. The path

```cpp
void dijkstra(int start, vector<long long>& dist, vector<vector<pair<int, int>>>& adj) {
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<pair<long long, int>>> pq;
    dist[start] = 0;
    pq.push({0, start});

    while (!pq.empty()) {

        auto top = pq.top();
        long long d = top.first;
        int u = top.second;
        pq.pop();

        if (d > dist[u]) continue;

        for (const auto& edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;

            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

```
int minCostPath(int g_nodes, vector<int> g_from, vector<int> g_to, vector<int> g_weight, int x, int y) {
    // Build adjacency list
    vector<vector<pair<int, int>>> adj(g_nodes + 1);
    int num_edges = g_from.size();
    for (int i = 0; i < num_edges; ++i) {
        int u = g_from[i];
        int v = g_to[i];
        int w = g_weight[i];
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    vector<long long> dist1(g_nodes + 1, LLONG_MAX);
    vector<long long> distx(g_nodes + 1, LLONG_MAX);
    vector<long long> disty(g_nodes + 1, LLONG_MAX);

    dijkstra(1, dist1, adj);
    dijkstra(x, distx, adj);
    dijkstra(y, disty, adj);

    if (dist1[x] == LLONG_MAX || distx[y] == LLONG_MAX || disty[g_nodes] == LLONG_MAX) {
        return -1;
    } else {
        long long total_cost = dist1[x] + distx[y] + disty[g_nodes];
        return static_cast<int>(total_cost);
    }
}
```

1. Valid BST Permutations

A binary tree is a data structure characterized by the following properties:

- It can be an empty tree where root = null.
- It can consist of a root node that contains a value and two sub-trees labeled "left" and "right".
- Each sub-tree also follows the definition of a binary tree.

A binary tree is a binary search tree (BST) if all the non-empty nodes follow these two rules:

- If a node has a left sub-tree, then all the values in its left sub-tree are smaller than its value.
- If a node has a right sub-tree, then all the values in its right sub-tree are greater than its value.

Given an integer, determine the number of valid BSTs that can be created by nodes numbered from 1 to that integer. Please see the samples below for diagrams based on 1, 2 or 3 nodes.

```cpp
int numTrees(int N) {
    const int mod = 1e8 + 7;
    vector<long long> dp(N + 1, 0);
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= N; ++i) {
        for (int j = 1; j <= i; ++j) {
            dp[i] = (dp[i] + ((dp[j - 1] % mod) * (dp[i - j] % mod)) % mod) % mod;
        }
    }
    return dp[N] % mod;
}

vector<int> numTreesForVector(const vector<int>& nums) {
    vector<int>ans;
    for (int n : nums) {
        ans.push_back(numTrees(n));
    }
    return ans;
}
```

```cpp
void numTrees(int N, vector < long long > & dp) {
    const int mod = 1e8 + 7;
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= N; ++i) {
        for (int j = 1; j <= i; ++j) {
            dp[i] = (dp[i] + ((dp[j - 1] % mod) * (dp[i - j] % mod)) % mod) % mod;
        }
    }
}

vector < int > numTreesForVector(const vector < int > & nums) {
    vector < int > ans;
    int mx = * max_element(begin(nums), end(nums));
    vector < long long > dp(mx + 1, 0);
    numTrees(mx, dp);
    for (int n: nums) {
        ans.push_back(dp[n]);
    }
    return ans;
}
```

1h 39m left

ALL

1 2 3 4 5 6 7 8 9

1. Binary Autocomplete

The programming interface for a legacy motor controller accepts commands as binary strings of variable length. The console has a very primitive autocomplete/autocorrect feature: as a new command is entered one character at a time, it will display the previously entered command that shares the *longest common prefix*. If multiple commands are tied, it displays the most recent one. If there is no previous command that shares a common prefix, it will display the most recent command.

Given a sequence of commands entered into the console, for each command, determine the index of the command last displayed by the autocomplete as it was entered. Return 0 if there is none.

Example

n = 6
command = ['000', '1110', '01', '001', '110', '11']

1. *'000' - 0* (No command has previously been entered)
2. *'1110'- 1* (There is no previous command that shares a common prefix, so the last command is shown)
3. *'01' - 1* (*'000'* shares the prefix *'0'* with the first command)
4. *'001' - 1* (*'000'* shares the prefix *'00'* with the first command)
5. *'110' - 2* (*'110'* shares the prefix *'11'* with the second command)
6. *'11' - 5* (*'11'* shares the prefix *'11'* with the fifth command most recently)

The return array is *[0, 1, 1, 1, 2, 5]*.

Function Description

Complete the function *autocomplete* in the editor below.

autocomplete has the following parameter(s):

string command[n]: an array of strings where *command[i]* denotes the $(i+1)^{th}$ entered command, $0 \leq i < n$.

Language: Java 17 | Environment

Autocomplete Loading...

```
1  import java.io.*;
14
15 class Result {
16
17     /*
18      * Complete the 'autocomplete' function below.
19      *
20      * The function is expected to return an INTEGER_ARRAY.
21      * The function accepts STRING_ARRAY command as parameter.
22      */
23
24     public static List<Integer> autocomplete(List<String>
   command) {
25     // Write your code here
26
27     }
28
29 }
30
31 public class Solution {
```

Test Results | Custom Input | Run Code | Run Tests | Submit

1h 38m left

Input Format For Custom Testing

Sample Case 0

Sample Input For Custom Testing

```
STDIN       Function
-----       --------
3      →    command[] size n = 3
100110 →    command = ['100110', '1001', '1001111']
1001
1001111
```

Sample Output

```
0
1
1
```

Explanation

1.*'100110'* - *0* (No command has previously been entered)
2.*'1001'* - *1* (The first command shares the prefix *'1001'*)
3. *'1001111'* - *1* (The first command shares the prefix *'10011'*)

Sample Case 1

Sample Input For Custom Testing

```
STDIN     Function
-----     --------
3    →    command[] size n = 3
1    →    command = ['1', '10', '11010']
10
11010
```

Sample Output

```
0
1
2
```

Connected to network.

Language: Java 17 | Environment | Autocomplete Loading...

```
rt java.io.*;
class Result {

    /*
     * Complete the 'autocomplete' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts STRING_ARRAY command as parameter.
     */

    public static List<Integer> autocomplete(List<String>
command) {
    // Write your code here

    }

}

public class Solution {
```

Test Results | Custom Input | Run Code | Run Tests | Submit

```cpp
vector<int> autocomplete(vector<string> command) {
    map<pair<int, int>, int> prefixMap;
    vector<int> result;

    for (int i = 0; i < command.size(); i++) {
        int binaryValue = 0;
        int lastIndex = i;

        for (int j = 0; j < command[i].size(); j++) {
            binaryValue = (binaryValue * 2) + (command[i][j] - '0');

            if (prefixMap.find({binaryValue, j + 1}) != prefixMap.end()) {
                lastIndex = prefixMap[{binaryValue, j + 1}] + 1;
            }

            prefixMap[{binaryValue, j + 1}] = i;
        }

        result.push_back(lastIndex);
    }

    return result;
}
```

1611. Minimum One Bit Operations to Make Integers Zero

Hard | Topics | Companies | Hint

Given an integer `n`, you must transform it into `0` using the following operations any number of times:

- Change the rightmost (`0th`) bit in the binary representation of `n`.
- Change the `ith` bit in the binary representation of `n` if the `(i-1)th` bit is set to `1` and the `(i-2)th` through `0th` bits are set to `0`.

Return *the minimum number of operations to transform* `n` *into* `0`.

```
3
4    int minimumOneBitOperations(int n) {
5        int t = 16;
6        while (t) {
7            n = n ^ (n >> t);
8            t /= 2;
9        }
10
11       return n;
12   }
13
```

I have been reading about Combinatorial Material for a while now.

I also solved a few examples. However am stuck at this one.

> Find the total number of ways a $3 \times n$ board can be painted using 3 colors while making sure no cells of the same row or the same column have entirely the same color. The answer must be computed modulo $10^9 + 7$.

I saw the solutions to $3 \times n$ using colors but all these had the constraint that no adjacent cells were to have the same color. There's no such constraint here.

So am just not sure how to do this given the center row can have a bunch of options as well.

```cpp
const int mod = 1e9 + 7;

int calc(int index, int a, int b, int c, int n, vector<vector<vector<vector<int>>>> &dp) {
    if (index == n) {
        // Check if any row has only one color
        if (a != 1 && a != 2 && a != 4 && b != 1 && b != 2 && b != 4 && c != 1 && c != 2 && c != 4)
            return 1;
        return 0;
    }
    if (dp[index][a][b][c] != -1) return dp[index][a][b][c];

    int ways = 0;
    for (int i = 0; i < 3; i++) {       // Color for row 1
        for (int j = 0; j < 3; j++) {   // Color for row 2
            for (int k = 0; k < 3; k++) { // Color for row 3
                if (i == j && j == k) continue; // Skip if all rows have the same color
                int new_a = a | (1 << i);
                int new_b = b | (1 << j);
                int new_c = c | (1 << k);
                ways = (ways + calc(index + 1, new_a, new_b, new_c, n, dp)) % mod;
            }
        }
    }
    return dp[index][a][b][c] = ways;
}

int countWays(int n) {
    vector<vector<vector<vector<int>>>> dp(
        n, vector<vector<vector<int>>>(8, vector<vector<int>>(8, vector<int>(8, -1))));
    return calc(0, 0, 0, 0, n, dp);
}
```

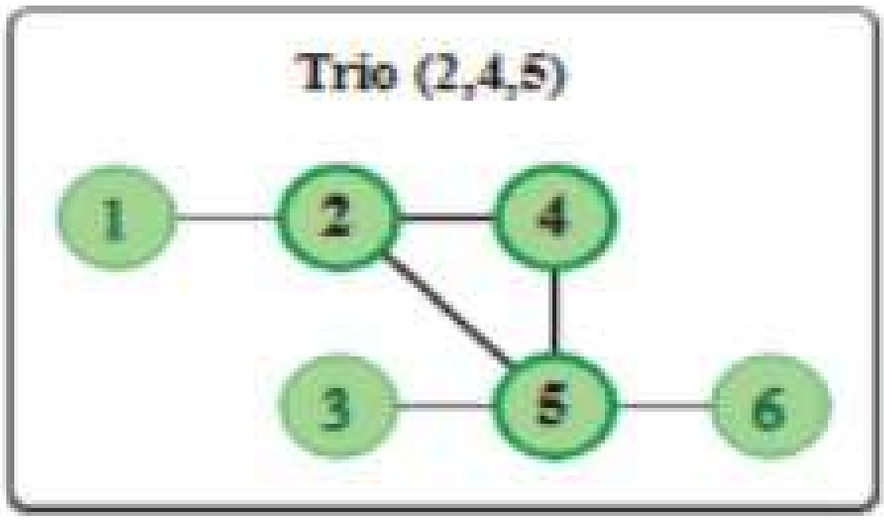
1. Social Media Connections

Social media connections can serve as a means of recognizing relationships among a group of people. These relationships can be represented as an undirected graph where edges join related people. A group of *n* social media friends is uniquely numbered from *1* to *friends_nodes*. The group of friends is expressed as a graph with *friends_edges* undirected edges, where each pair of *best friends* is directly connected by an edge. A *trio* is defined as a group of three best friends. The *friendship score* for a person in a trio is defined as the number of best friends that person has outside of the trio. The *friendship sum* for a trio is the sum of the trio's friendship scores.

Given friendship connection data, create an undirected graph and determine the minimum friendship sum for all trios of best friends in the group. If no such trio exists, return *-1*.

Example

friend_nodes = 6
friends_from = [1, 2, 2, 3, 4, 5]
friends_to = [2, 4, 5, 5, 5, 6]



Friend	Best Friends
1	2
2	1, 4, 5
3	5
4	2, 5
5	2, 3, 4, 6
6	5

A graph of *n = 6* friends where the only trio of *best friends* is *(2, 4, 5)*.

The friendship scores based on the graph above are:

Friend	Outside Friends	Which Friends Are Outside
2	1	1
4	0	
5	2	3, 6

In the diagram above, the total friendship score is *1 + 0 + 2 = 3* for the trio *(2, 4, 5)*.

```cpp
int bestTrio(int friends_nodes, int friends_edges, vector<int>& friends_from,
    vector<int>& friends_to) {
    vector<set<int>> adj(friends_nodes + 1);
    vector<int> degrees(friends_nodes + 1, 0);

    for (int i = 0; i < friends_edges; ++i) {
        int u = friends_from[i];
        int v = friends_to[i];
        adj[u].insert(v);
        adj[v].insert(u);
        degrees[u]++;
        degrees[v]++;
    }

    int min_friendship_sum = INT_MAX;

    for (int i = 1; i <= friends_nodes; ++i) {
        for (int j : adj[i]) {
            if (j > i) {
                for (int k : adj[i]) {
                    if (k > j) {
                        if (adj[j].count(k)) {
                            int friendship_sum = (degrees[i] - 2) +
                                (degrees[j] - 2) + (degrees[k] - 2);
                            min_friendship_sum = min(min_friendship_sum,
                                friendship_sum);
                        }
                    }
                }
            }
        }
    }

    return min_friendship_sum == INT_MAX ? -1 : min_friendship_sum;
}
```

27. Preemptive Scheduling

When multiple tasks are executed on a single-threaded CPU, the tasks are scheduled based on the principle of pre-emption. When a higher-priority task arrives in the execution queue, then the lower-priority task is pre-empted, i.e. its execution is paused until the higher-priority task is complete.

There are *n* functions to be executed on a single-threaded CPU, with each function having a unique ID between 0 and *n* - 1. Given an integer *n*, representing the number of functions to be executed, and an execution log as an array of strings, *logs*, of size *m*, determine the *exclusive times* of each of the functions. Exclusive time is the sum of execution times for all calls to a function. Any string representing an execution log is of the form *{function_id}:{"start" | "end"}:{timestamp}*, indicating that the function with ID = *function_id*, either starts or ends at a time identified by the *timestamp* value.

Note: While calculating the execution time of a function call, both the starting and ending times of the function call have to be included. The log of the form *{function_id}:{start}:{timestamp}* means that the running function is preempted at the beginning of *timestamp* second. The log of the form *{function_id}:{end}:{timestamp}* means that the

```cpp
struct Log {
    int id;
    string status;
    int timestamp;
};
vector<int> getTotalExecutionTime(int n, vector<string> &logs) {
    vector<int> times(n, 0);
    stack<Log> st;
    for (const string &log : logs) {
        stringstream ss(log);
        string temp, status;
        int timestamp;
        getline(ss, temp, ':');
        getline(ss, status, ':');
        ss >> timestamp;
        Log item = {stoi(temp), status, timestamp};
        if (item.status == "start") {
            st.push(item);
        } else {
            assert(st.top().id == item.id);
            int time_added = item.timestamp - st.top().timestamp + 1;
            times[item.id] += time_added;
            st.pop();
            if (!st.empty()) {
                assert(st.top().status == "start");
                times[st.top().id] -= time_added;
            }
        }
    }
    return times;
}
```

Challenge

Claire is mentoring a group of students to participate in the upcoming Women's Day Mathematics Challenge. In one of her classes, she plans to teach modulo arithmetic to the girls.

She takes an array of numbers and asks the girls to perform the following operation: pick two adjacent **positive** numbers a_i and a_{i+1}, and replace the two numbers with either $(a_i \% a_{i+1})$ or $(a_{i+1} \% a_i)$, where $x \% y$ denotes x modulo y. Thus, after each operation, the length of the array decreases by 1.

Claire asks the girls to find the minimum possible length of the array which can be achieved by performing the above operation any number of times.

Example

Consider, the length of the array to be $n = 4$, and the array of numbers to be $arr = [1, 1, 2, 3]$.
The following sequence of operations can be performed (considering 1-based indexing):

1. arr = [1, 1, 2, 3]: Choose i = 3, thus arr[i] = 2 and arr[i + 1] = 3. We get the new value to be given by 2 %·3 = 2 or 3 % 2 = 1. The resulting array can thus be [1, 1, 2] or [1, 1, 1]. We consider the former one to minimize the array length.

1. arr = [1, 1, 2, 3]: Choose i = 3, thus arr[i] = 2 and arr[i + 1] = 3. We get the new value to be given by 2 % 3 = 2 or 3 % 2 = 1. The resulting array can thus be [1, 1, 2] or [1, 1, 1]. We consider the former one to minimize the array length.

2. arr = [1, 1, 2]: Choose i = 2, thus arr[i] = 1 and arr[i + 1] = 2. We can get the new array as [1, 1].

3. arr = [1, 1]: Choose i = 1 to get the array [0].

Thus the minimum possible length for the above array would be 1.

Function Description

Complete the function *getMinLength* in the editor below.

getMinLength has the following parameter:

int arr[arr[0],...arr[n-1]]: the given array of integers

Returns

int: the minimum possible length of the array.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq arr_i \leq 10^9$

The first line contains an integer, n, denoting the number of elements in arr.

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer describing arr_i.

▼ Sample Case 0

Sample Input For Custom Testing

STDIN		FUNCTION
4	→	arr[] size n = 4
2	→	arr = [2, 2, 2, 2]
2		
2		
2		

Sample Output

```
2
```

Explanation

We can choose i = 1 first to get [0, 2, 2]. Now we can choose i = 2 to get [0, 0]. Since there are no two adjacent positive numbers, we cannot perform any further operation. Thus the minimum possible length of the array is 2.

▼ Sample Case 1

Sample Input For Custom Testing

STDIN	FUNCTION

minimum possible length of the array is 2.

▼ Sample Case 1

Sample Input For Custom Testing

```
STDIN           FUNCTION
-----           --------
3         →     arr[] size n = 3
1         →     arr = [1, 1, 3]
1
3
```

Sample Output

```
1
```

Explanation

We choose i = 2 to get [1, 1]. Now we can choose i = 1 to get [0] which has the minimum possible length of 1.

```
int m = *min_element(arr.begin(), arr.end());
    int c = 0;
    for (int x : arr) {
        if (x % m) {
            return 1;
        }
        if (x == m) {
            ++c;
        }
    }
    return (c + 1) / 2;
```

Women's day mathematics challenge

C++20

1. String Subsequences

Given two strings, determine the number of times the first one appears as a subsequence in the second one. A subsequence is created by eliminating any number of characters from a string (possibly 0) without changing the order of the characters retained.

Example

s1 = "ABC"
s2 = "ABCBABC"

```
int count_subsequences(const string & s1,
    const string & s2)
{
    int m = s1.length(), n = s2.length();
    vector < vector < int >> dp(m + 1, vector < int > (n + 1, 0));
    for (int j = 0; j <= n; ++j)
    {
        dp[0][j] = 1;
    }
    for (int i = 1; i <= m; ++i)
    {
        for (int j = 1; j <= n; ++j)
        {
            if (s1[i - 1] == s2[j - 1])
            {
                dp[i][j] = dp[i][j - 1] + dp[i - 1][j - 1];
            }
            else
            {
                dp[i][j] = dp[i][j - 1];
            }
        }
    }
    return dp[m][n];
}
```

1. Quiz Competition

A class has a number of students, each with a particular area of knowledge, or *talent*. Each talent is represented as an integer from *1* to *talentsCount*. Teams must be formed for a quiz competition, and must have at least one member with each of the talents. The talents are listed in an array, and the students must be chosen consecutively, starting at any element index. For each starting index, determine the minimum number of students that must be selected to have at least one team member with each talent. If it is not possible, return -1 for that index.

Example

talentsCount = 3

talent = [1, 2, 3, 2, 1]

There is no way fewer than *3* students can have at least *3* talents. The following subarrays of length *3* or more are possible:

Starting at position 1: ***[1, 2, 3]***, *[1, 2, 3, 2]*, *[1, 2, 3, 2, 1]*

Starting at position 2: *[2, 3, 2]*, ***[2, 3, 2, 1]***

Starting at position 3: ***[3, 2, 1]***

No further subarrays of length 3 or more exist.

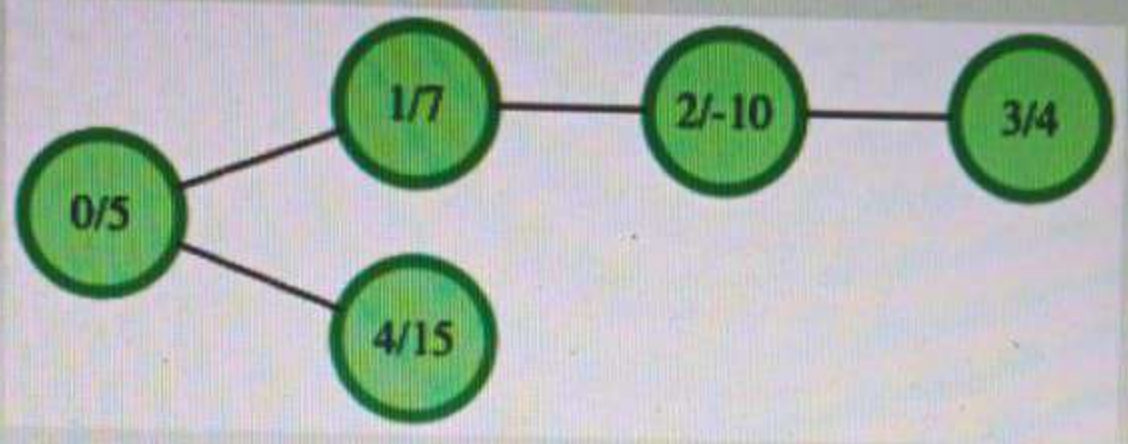
```cpp
vector < int > findMinTeamLengths(int tCount, vector < int > & tList)
{
    int n = tList.size();
    vector < int > res(n,
        -1);
    unordered_map < int, int > tMap;
    int dCount = 0;
    int l = 0;
    for (int r = 0; r < n; r++)
    {
        int curT = tList[r];
        if (tMap[curT] == 0)
        {
            dCount++;
        }
        tMap[curT]++;
        while (dCount == tCount)
        {
            res[l] = r - l + 1;
            int lT = tList[l];
            tMap[lT]--;
            if (tMap[lT] == 0)
            {
                dCount--;
            }
            l++;
        }
    }

    return res;
}
```

ALL

1. Best Sum Any Tree Path

Given a tree rooted at node *0* and a *value* assigned to each node, determine the maximum sum of the values along any path in the tree. The path must not be empty, and in some cases it might not go through the root. The following tree (labeled *node number/value*) is analyzed below.



Two of the possible paths are $4 \rightarrow 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ which has a sum of $15 + 5 + 7 + -10 + 4 = 21$ and $1 \rightarrow 2 \rightarrow 3$ with a sum of $7 + -10 + 4 = 1$. A third possible path, the maximum sum path, is $4 \rightarrow 0 \rightarrow 1$ with a sum of $15 + 5 + 7 = 27$.

Function Description

Complete the function *bestSumAnyTreePath* in the editor below. It must return an integer that represents the largest value sum on any non-empty path in the tree.

bestSumAnyTreePath has the following parameter(s):

- *parent int[]*: integer array where *parent[i] = j* means that node *j* is a parent of node *i*. *parent[0]* will be set to *-1* to indicate that node *0* is the root of the tree.
- *values int[]*: integer array where *values[i]* denotes the value of node *i*

```cpp
int dfs(int node, vector < vector < int >> & g, vector < int > & values, int & ans)
{
    int current = values[node];
    int max_upward = 0;
    for (int child: g[node]) {
        int child_sum = dfs(child, g, values, ans);
        max_upward = max(max_upward, child_sum);
    }
    int max_including_node = max(current, current + max_upward);
    ans = max(ans, max_including_node);

    int sum_from_children = 0;

    if (!g[node].empty()) {
        vector < int > ch;
        for (int child: g[node])
            ch.push_back(dfs(child, g, values, ans));

        sort(ch.rbegin(), ch.rend());
        sum_from_children = current + ch[0];

        if (ch.size() > 1)
            sum_from_children += ch[1];

        ans = max(ans, sum_from_children);
    }
    return max_including_node;
}
```

```cpp
int bestSumAnyTreePath(vector < int > parent, vector < int > values)
{
    int n = parent.size();
    vector < vector < int >> g(n);
    int root = -1;
    for (int i = 0; i < n; i++) {
        if (parent[i] == -1)
            root = i;
        else
            g[parent[i]].push_back(i);
    }
    int ans = INT_MIN;
    dfs(root, g, values, ans);
    return ans;
}
```

Determine if a triangle formed by three points a(x1, y1), b(x2, y2), and c(x3, y3) is non-degenerate. If two given points, p = (xp, yp) and q = (xq, yq), are located on or inside a triangle, they *belong* to the triangle. Return the correct scenario number.

Scenarios:

- 0: the lines do not form a valid non-degenerate triangle
- 1: point *p* belongs to the triangle but point *q* does not
- 2: point *q* belongs to the triangle but point *p* does not
- 3: both points *p* and *q* belong to the triangle
- 4: neither point *p* nor point *q* belong to the triangle

Note: A triangle is considered non-degenerate if it meets the following conditions, where |ab| denotes the length of the line segment between points a and b:

- $|ab| + |bc| > |ac|$
- $|bc| + |ac| > |ab|$
- $|ab| + |ac| > |bc|$

Example

```
double area(int x1, int y1, int x2, int y2, int x3, int y3)
{
    return abs((x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2)) / 2.0);
}

bool isInside(int x1, int y1, int x2, int y2, int x3, int y3, int x, int y)
{
    double A = area(x1, y1, x2, y2, x3, y3);
    double A1 = area(x, y, x2, y2, x3, y3);
    double A2 = area(x1, y1, x, y, x3, y3);
    double A3 = area(x1, y1, x2, y2, x, y);

    return (A == A1 + A2 + A3);
}

int dotheybelong(int x1, int y1, int x2, int y2, int x3, int y3, int xp, int yp, int xq, int yq) {
    int tri_area = area(x1, y1, x2, y2, x3, y3);
    if (tri_area == 0) {
        return 0;
    }
    bool ptru = isInside(x1, y1, x2, y2, x3, y3, xp, yp);
    bool qtru = isInside(x1, y1, x2, y2, x3, y3, xq, yq);

    //based on results return answer
}
```

1. Rearrange Students

In your school, two lines of students are formed A and B, with each line having exactly N students. The two lines of students will face each other. The Physical Education teacher wants the heights of all students standing across from each other to be equal. To do this, students may be swapped from one line to another. Any student in one line can be swapped with any student in the other line. Each swap counts as one operation, and each operation costs the minimum height in the swap. Reordering the students within one line has no cost. Determine the minimal cost to achieve the desired result. If it is impossible, return -1.

Example

arrA = [4, 2, 2, 2]

arrB = [1, 4, 1, 2]

...ordering, they are *arrA*

```cpp
long long minCost(vector<int> &a, vector<int> &b) {
    map<int, int> mp;
    int n = a.size();
    int mini = INT_MAX;

    for (int i = 0; i < n; i++) {
        mp[a[i]]++;
        mp[b[i]]--;
        mini = min(mini, a[i]);
        mini = min(mini, b[i]);
    }

    vector<int> x;
    for (auto it : mp) {
        int t = it.second;
        if (t % 2 != 0) return -1;
        for (int i = 0; i < abs(t) / 2; i++) {
            x.push_back(it.first);
        }
    }

    sort(x.begin(), x.end());
    long long ans = 0;
    int m = x.size();
    for (int i = 0; i < m / 2; i++) {
        ans += min(x[i], 2 * mini);
    }

    return ans;
}
```

1. Get the Groups

As new students begin to arrive at college, each receives a unique ID number, *1* to *n*. Initially, the students do not know one another, and each has a different circle of friends. As the semester progresses, other groups of friends begin to form randomly.

There will be three arrays, each aligned by an index. The first array will contain a *queryType* which will be either *Friend* or *Total*. The next two arrays, *students1* and *students2*, will each contain a student ID. If the query type is *Friend*, the two students become friends. If the query type is *Total*, report the sum of the sizes of each group of friends for the two students.

Example

n = 4

queryType = ['Friend', 'Friend', 'Total']

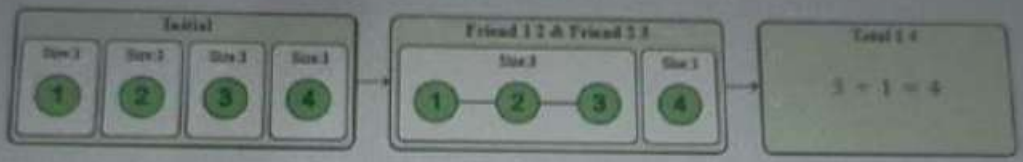
Example

n = 4

queryType = ['Friend', 'Friend', 'Total']

student1 = [1, 2, 1]

student2 = [2, 3, 4]



The queries are assembled, aligned by index:

Index	queryType	student1	student2
0	Friend	1	2
1	Friend	2	3
2	Total	1	4

Students will start as discrete groups *{1}*, *{2}*,*{3}* and *{4}*. Students *1* and *2* become friends with the first query, as well as students *2* and *3* in the second. The new groups are *{1, 2}*, *{2, 3}* and *{4}* which simplifies to *{1, 2, 3}* and *{4}*. In the third query, the number of friends for student *1* = *3* and student *4* = *1* for a Total = *4*. Notice that student *3* is indirectly part of the circle of friends of student *1* because of student *2*.

```cpp
vector<int> parent, sz;

int find(int x) {
    if (parent[x] != x) parent[x] = find(parent[x]);
    return parent[x];
}

void unite(int x, int y) {
    int px = find(x), py = find(y);
    if (px != py) {
        if (sz[px] < sz[py]) swap(px, py);
        parent[py] = px;
        sz[px] += sz[py];
    }
}

vector<int> getTheGroups(int n, vector<string>& q, vector<int>& f1, vector<int>& f2) {
    parent.resize(n + 1);
    sz.resize(n + 1, 1);
    for (int i = 0; i <= n; ++i) parent[i] = i;

    vector<int> res;
    for (int i = 0; i < q.size(); ++i) {
        if (q[i] == "Friend") {
            unite(f1[i], f2[i]);
        } else if (q[i] == "Total") {
            int p1 = find(f1[i]), p2 = find(f2[i]);
            if (p1 == p2) {
                res.push_back(sz[p1]);
            } else {
                res.push_back(sz[p1] + sz[p2]);
            }
        }
    }
    return res;
}
```

1. Intelligent Substring

ALL

There are two types of characters in a particular language: special and normal. A character is *special* if its value is *1* and *normal* if its value is *0*. Given string s, return the longest substring of *s* that contains at most *k* normal characters. Whether a character is normal is determined by a 26-digit bit string named *charValue*. Each digit in *charValue* corresponds to a lowercase letter in the English alphabet.

Example

s = 'abcde'

alphabet = abcdefghijklmnopqrstuvwxyz

charValue = 10101111111111111111111111

For clarity, the alphabet is aligned with charValue below:

```
 alphabet = abcdefghijklmnopqrstuvwxyz
charValue = 10101111111111111111111111
```

The only normal characters in the language (according to *charValue*) are *b* and *d*. The string *s* contains both of these characters. For *k = 2*, the longest substring of *s* that contains at most *k = 2* normal characters is *5* characters long, *abcde*, so the return value is *5*. If *k = 1* instead, then the

```
int getSpecialSubstring(string s, int k, string charValue){
    int n = s.length(), res = 0, c = 0, l = 0;
    for (int r = 0; r < n; ++r) {
        if (charValue[s[r] - 'a'] == '0') c++;
        while (c > k) {
            if (charValue[s[l] - 'a'] == '0') c--;
            l++;
        }
        res = max(res, r - l + 1);
    }
    return res;
}
```

1. Element Swapping

A software development firm is hiring engineers and used the following challenge in its online test.

Given an array *arr* that contains *n* integers, the following operation can be performed on it any number of times (possibly zero):

- Choose any index *i* ($0 \le i < n - 1$) and swap *arr[i]* and *arr[i + 1]*.
- Each element of the array can be swapped at most once during the whole process.

The strength of an index *i* is defined as *(arr[i] * (i + 1))*, using 0-based indexing. Find the maximum possible sum of the strength of all indices after optimal swaps. Mathematically, maximize the following.

$$\sum_{i=0}^{n-1} \text{arr}[i] \times (i+1)$$

Example

Consider *n = 4, arr = [2, 1, 4, 3]*.

It is optimal to swap *(arr[2], arr[3])* and *(arr[0], arr[1])*. The final array is [1, 2, 3, 4]. The sum of strengths = (1 * 1 + 2 * 2 + 3 * 3 + 4 * 4) = 30, which is maximum possible. Thus, the answer is 30.

Function Description

Complete the function *getMaximumSumOfStrengths* in the editor below.

getMaximumSumOfStrengths has the following parameter:

int arr[n]: the initial array

Returns

long int: the maximum possible sum of strengths of all indices after the operations are applied optimally

Constraints

- $1 \le n \le 10^5$
- $1 \le arr[i] \le 10^5$

▶ **Input Format For Custom Testing**

```cpp
long long getMaximumStrength(vector < int > a)
{
    int n = a.size();
    vector < vector < int >> dp(n, vector < int > (2));
    dp[0][0] = a[0];
    dp[0][1] = a[0];
    for (long long i = 1; i < n; i++)
    {
        dp[i][0] = max(dp[i - 1][0], dp[i - 1][1]) + a[i] * (i + 1);
        dp[i][1] = a[i] * i + a[i - 1] * (i + 1) + (i - 2 >= 0 ? max(dp[i - 2][0], dp[i - 2][1]) : 0);
    }
    return max(dp[n - 1][0], dp[n - 1][1]);
}
```

1. Horizontal Pod Autoscaler

The developers are trying to optimize their horizontal pod autoscaler for their micro-services. There are *n* micro-services where the number of pods for the i^{th} micro-service is *pods[i]*. According to traffic, the number of pods of a service can increase or decrease. Also, at specific times when there is expected traffic, all services with fewer than *x* pods are assigned *x* pods.
There is an event log of size *m*, which is described as a 2D array *logs* where *logs[i]* is an array of integers of size = 3. The interpretation of these logs are shown.

- [1, p, x]: the number of pods of the p^{th} micro-service is changed to *x (1 ≤ p ≤ n)*.
- [2, -1, x]: all the micro-services whose number of pods is less than *x are* changed to *x*.

Find the resulting number of pods for the micro-services.

Example

Given *n* = 4, *pods* = [2, 4, 1, 4], *m* = 3, *logs* = *[[1, 2, 30], [1, 3, 4], [2, -1, 10]]*.

Current Number of Pods	Log	Resulting Pods

Language: Python 3

```
#!/bin/python
#
# Complete th
#
# The functio
# The functio
#  1. INTEGER
#  2. 2D_INTE
#

def findPodCo
    # Write y

if __name__ =
```

Test Results

Search

1h 59m left

ALL

Find the resulting number of pods for the micro-services.

Example

Given $n = 4$, $pods = [2, 4, 1, 4]$, $m = 3$, $logs = [[1, 2, 30], [1, 3, 4], [2, -1, 10]]$.

Current Number of Pods	Log	Resulting Pods
[2, 4, 1, 4]	[1, 2, 30] 2nd member's pod count is changed to 30.	[2, 30, 1, 4]
[2, 30, 1, 4]	[1, 3, 4] 3rd member's pod count is changed to 4.	[2, 30, 4, 4]
[2, 30, 4, 4]	[2, -1, 10] All the micro-services with fewer than 10 pods are changed to 10.	[10, 30, 10, 10]

The final number of pods is [10, 30, 10, 10].

Function Description

Complete the function findPodCount in the editor

Language: Python 3

```
#!/bin/pyth
#
# Complete
#
# The funct
# The funct
#  1. INTE
#  2. 2D_I
#

def findPo
    # Writ


if __name_
```

Test Results

Search

```cpp
vector < int > findPodCount(int n, vector < int > & pods, const vector < vector < int >> & logs) {
    int mx = 0;

    for (const auto & log: logs) {
        int type = log[0];
        int p = log[1];
        int x = log[2];

        if (type == 1) {
            if (p >= 1 && p <= n) {
                pods[p - 1] = x;
            }
        } else if (type == 2) {
            mx = max(mx, x);
        }
    }

    for (int i = 0; i < n; ++i) {
        pods[i] = max(pods[i], mx);
    }

    return pods;
}
```

1. K-Means Clustering

In a k-means clustering problem, a dataset contains n data points, where i^{th} data point is represented by the feature vector *location[i]*. The goal is to create k clusters, where the cluster centers or the cluster centroids can be placed at any point in the feature space. The overall quality of the clustering is measured by the maximum distance between any data point and its nearest cluster center.

The best possible quality is achieved by optimally placing the cluster centers to minimize this maximum distance. Determine this maximum distance between any data point and its nearest cluster center.

Note: The distance between two feature points x and y is defined as $|x - y|$, where $|x|$ denotes the absolute value of x.

Example

$n = 5$

$location = [4, 1, 6, 7, 2]$

$k = 2$

```cpp
int getMaximumDistance(vector<int> location, int k) {
    int n = location.size();
    sort(location.begin(), location.end());
    int maxi = location[n - 1] - location[0];
    int left = 0, right = maxi;

    while (left < right) {
        int mid = left + (right - left) / 2;
        int center = 1;
        int prev = location[0];

        for (int i = 1; i < n; i++) {
            if (location[i] - prev > mid * 2) {
                center++;
                prev = location[i];
            }
        }

        if (center <= k) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return left;
}
```

Minimize Max Distance to Gas Station

Difficulty: **Hard** Accuracy: **38.36%** Submissions: **62K+** Points: **8**

We have a horizontal number line. On that number line, we have gas **stations** at positions stations[0], stations[1], ..., stations[N-1], where **n** = size of the stations array. Now, we add **k** more gas stations so that **d**, the maximum distance between adjacent gas stations, is minimized. We have to find the smallest possible value of d. Find the answer **exactly** to 2 decimal places.

Example 1:

Input:
n = 10
stations = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
k = 9
Output: 0.50
Explanation: Each of the 9 stations can be added mid way between all the existing adjacent stations.

```cpp
double findSmallestMaxDist(vector<int> &stas, int k) {
    // Code here
    int n=stas.size();
    double low=0,high=stas[n-1]-stas[0];
    while(high-low>1e-6){
        double mid=low+(high-low)/2.0;
        if(val(stas,k,n,mid))high=mid;
        else low=mid;
    }
    return low+0.000001;
}
bool val(vector<int>&stas,int k, int n,double dis){
    int ns=0;
    for(int i=1;i<n;++i){
        ns+=floor((stas[i]-stas[i-1])/dis);
    }
    return ns<=k;
}
```